Investigación sobre la aplicación de tokens JWT en GameVault

1. Introducción

La autenticación es un mecanismo que permite verificar la identidad de los usuarios que acceden a un sistema. En el proyecto GameVault, desarrollado con Node.js, Express, PostgreSQL y React, se implementará un sistema de autenticación basado en JSON Web Tokens (JWT).

El objetivo es permitir que los usuarios se registren, inicien sesión y accedan a las funcionalidades del sistema de inventario únicamente después de haber sido autenticados.

La API emitirá un token al iniciar sesión correctamente. El frontend utilizará ese token para identificar al usuario en las solicitudes posteriores al backend.

2. ¿Qué es un JWT?

JSON Web Token (JWT) es un estándar utilizado para transmitir información entre dos partes mediante un objeto JSON. En el contexto de la autenticación, se utiliza para representar la identidad de un usuario y comprobar que la información contenida en el token no ha sido modificada.

Un JWT firmado está compuesto por tres partes:

- **Header:** contiene información sobre el tipo de token y el algoritmo utilizado para firmarlo.
- **Payload:** contiene los datos o declaraciones (claims) asociados al token, como el identificador del usuario y las fechas de emisión y expiración.
- **Signature:** permite verificar la autenticidad e integridad del token mediante una clave secreta o una clave criptográfica.

Las tres partes se separan mediante puntos:

```
header.payload.signature
```

Es importante aclarar que un JWT firmado no está necesariamente cifrado. Su contenido puede ser leído por quien tenga el token, por lo que no deben incluirse contraseñas, información confidencial ni datos sensibles.

3. Generación y validación de JWT en Node.js

En Node.js se puede utilizar la biblioteca `jsonwebtoken` para crear y verificar tokens.

La generación se realiza mediante `jwt.sign()`, que recibe el contenido del token, una clave secreta y opciones como el tiempo de expiración.

La validación se realiza mediante `jwt.verify()`. Esta función comprueba la firma y puede verificar la expiración del token. Si el token es inválido o ha expirado, se produce un error que debe ser manejado por el backend.

En GameVault se utilizará la biblioteca `jsonwebtoken` junto con Express para emitir tokens durante el inicio de sesión y validarlos mediante un middleware antes de permitir el acceso a las rutas protegidas.

4. Tiempo de expiración y renovación del token

Los JWT deben tener un tiempo de expiración para limitar el período durante el cual pueden utilizarse.

La declaración `exp` representa la fecha y hora a partir de la cual el token deja de ser válido. La biblioteca `jsonwebtoken` permite establecer este límite mediante la opción `expiresIn`.

Para el proyecto se propone utilizar un tiempo de expiración de **1 hora**. Este valor es apropiado para una demostración académica y permite comprobar el comportamiento de la aplicación cuando un token vence.

Existen dos tipos de tokens que pueden utilizarse en sistemas más avanzados:

- **Access token:** se utiliza para acceder a los recursos protegidos y normalmente tiene una duración corta.
- **Refresh token:** permite obtener un nuevo access token sin solicitar nuevamente las credenciales del usuario.

La renovación de tokens requiere mecanismos adicionales de seguridad, como controlar la validez y revocación de los refresh tokens. Para el alcance del Día 2 de GameVault, no se implementará un sistema de refresh tokens. Cuando el access token expire, el usuario deberá iniciar sesión nuevamente.

5. Almacenamiento del token en React

El frontend debe almacenar el token recibido del backend y enviarlo en las solicitudes posteriores.

Existen diferentes alternativas:

localStorage

Permite almacenar información en el navegador de forma persistente, incluso después de cerrar y abrir la página.

Su principal ventaja es la facilidad de implementación. Sin embargo, el token puede ser leído por código JavaScript que se ejecute en el mismo origen. Por ello, una vulnerabilidad de Cross-Site Scripting (XSS) podría permitir que un atacante obtenga el token.

sessionStorage

Almacena información durante la sesión de una pestaña del navegador. Su contenido se elimina cuando la pestaña o ventana se cierra.

Reduce la persistencia del token, pero sigue siendo accesible desde JavaScript y, por tanto, también está expuesto ante vulnerabilidades XSS.

Cookies HttpOnly

Las cookies configuradas con `HttpOnly` no pueden ser leídas directamente mediante JavaScript. Si también se utilizan las opciones `Secure` y `SameSite`, pueden ofrecer una protección adicional.

No obstante, el uso de cookies requiere considerar la protección contra Cross-Site Request Forgery (CSRF), ya que el navegador puede enviar las cookies automáticamente en determinadas solicitudes.

Decisión para GameVault

Para cumplir con los requisitos académicos y mantener una integración sencilla con React y Axios, se utilizará `localStorage` para almacenar el access token.

Esta decisión facilita la implementación del contexto de autenticación, las rutas protegidas y el cierre de sesión. Sin embargo, se reconoce que `localStorage` no es la alternativa más segura para aplicaciones de producción.

Se evitará almacenar información sensible en el token y se aplicarán buenas prácticas para reducir el riesgo de XSS.

6. Middleware de autenticación en Express

El middleware de autenticación es una función que se ejecuta antes de que una solicitud llegue al controlador de una ruta protegida.

Su funcionamiento consiste en:

1. Leer el encabezado `Authorization` de la solicitud.
2. Comprobar que tenga el formato `Bearer <token>`.
3. Extraer el token.
4. Validar su firma y expiración mediante `jwt.verify()`.
5. Obtener el identificador del usuario autenticado.
6. Permitir que la solicitud continúe si el token es válido.

Si el token no existe, es inválido o ha expirado, el middleware debe responder con el código HTTP `401 Unauthorized`.

En el proyecto GameVault, las rutas de las entidades del inventario estarán protegidas. Las rutas de registro e inicio de sesión serán públicas, mientras que la ruta del perfil actual requerirá un token válido.

7. Envío del token desde React

El frontend utilizará Axios para comunicarse con la API.

Después de iniciar sesión correctamente, el token se almacenará en `localStorage`. Posteriormente, Axios enviará el token mediante el encabezado:

```
Authorization: Bearer <token>
```

Se utilizará una instancia común de Axios para centralizar la configuración de la API y agregar automáticamente el token a las solicitudes.

Si el backend responde con el código `401`, el frontend eliminará el token almacenado y redirigirá al usuario a la pantalla de inicio de sesión.

El cierre de sesión también eliminará el token y la información del usuario del estado de autenticación.

8. Buenas prácticas de seguridad

Para implementar la autenticación de GameVault se tomarán en cuenta las siguientes recomendaciones:

- Las contraseñas se almacenarán mediante un algoritmo de hashing seguro, como bcrypt, nunca en texto plano.
- La clave secreta del JWT se almacenará en el archivo `.env`.
- El archivo `.env` no se incluirá en el repositorio de GitHub.
- Se utilizará un archivo `.env.example` sin valores secretos reales.
- El token tendrá un tiempo de expiración definido.
- El payload del JWT contendrá únicamente la información necesaria para identificar al usuario.
- No se incluirán contraseñas ni información sensible en el payload.
- El backend verificará la firma del token antes de permitir el acceso a las rutas protegidas.
- En un entorno de producción se utilizará HTTPS para proteger las comunicaciones.
- Se validarán los datos recibidos desde el frontend en el backend.

9. Conclusión

La implementación de JWT permitirá proteger la API de GameVault y controlar el acceso a las operaciones del inventario.

El uso de bcrypt protegerá las contraseñas almacenadas, mientras que el middleware de Express verificará la autenticidad de las solicitudes.

La integración con React y Axios permitirá mantener la sesión del usuario durante el uso de la aplicación, proteger las páginas privadas y gestionar el cierre de sesión.

La solución propuesta cumple con los requisitos de autenticación establecidos para el Día 2 del mini proyecto, manteniendo una arquitectura sencilla y adecuada para el alcance académico.

10. Fuentes consultadas

1. OWASP Foundation. *JSON Web Token Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_Cheat_Sheet.html
2. Auth0. *node-jsonwebtoken — Documentation*. https://github.com/auth0/node-jsonwebtoken
3. OWASP Foundation. *HTML5 Security Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html
4. OWASP Foundation. *Session Management Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
5. OWASP Foundation. *Web Security Testing Guide — Testing for JSON Web Tokens*. https://owasp.org/www-project-web-security-testing-guide/